Computabilidade (INF1015)
2023-1 - Lista II
Prof. Edward Hermann Haeusler

1. Foi demonstrado durante o curso que o conjunto das funções Turing Computáveis está contido no conjunto das funções parcialmente recursivas. Isto é, para toda Máquina de Turing que computa uma função parcial F de N em N, esta é F é parcialmente recursiva. Uma consequência desta demonstração é que toda função Turing computável pode ser representada em linguagem de funções parciais, com no máximo uma minimização ilimitada. Explique porque este fato, também conhecido como teorema da formal normal de Kleene, demonstra que toda função computável pode ser escrita com no máximo um comando "while", da forma "While v do cmds end". Resposta deve ser sucinta e capaz de convencer um colega seu que não fez o curso.

2. Mostre como obter a prova do teorema da forma normal de Kleene em termos de lambda-calculus, isto é, toda função lambda-definível pode ser definida por um λ-termo que tem somente uma ocorrência do operador $\Omega = (\lambda h.(\lambda x.h(xx))(\lambda x.h(xx)))$. Obs: Você pode precisar representar recursão primitiva sem usar o Ω.

3. Mostre como definir, usando o tipo de dado para parificaçao [_,_], o tipo de dado de listas de aridade arbitrária.

4. Defina em λ-calculus as funções, $f(n) = \sum_{i=0,n} i$, $fib(n)$= "o n-ésimo número de Fibonacci" e o número de combinações de n elementos em conjuntos de k elementos.

5. A função de Ackerman é definida pelas sequintes equações recorrentes:

$$A(0,n) = n+1 \quad (1)$$
$$A(m+1,0) = A(m,1) \quad (2)$$
$$A(m+1,n+1) = A(m,A(m+1,n)) \quad (3)$$

   Utilize o teorema da recursão para mostrar que A é lambda-computável.

6. Uma função $F:\mathbb{N}^n \to \mathbb{N}$ definida por recorrência no estilo das funções gerais recursivas de Kleene é um sistema de equações na forma abaixo, onde cada vetor $\vec{x}$ é a lista $x_1,\ldots,x_n$, e cada t^i_j é Suc, Z e P^m_l, i.e, as funções sucessor, zero e projeção. Os termos E_i são formados por composição das funções básicas Suc, Z e P^i_j com possivelmente a função F, daí usarmos a notação $E_i(F)\vec{x}$. Em geral pode-se incluir uma função G já definida por recursão geral.

$$\begin{aligned} F \circ \langle t^1_1,\ldots,t^1_n\rangle\vec{x} &= E_1(F)\vec{x} \\ F \circ \langle t^2_1,\ldots,t^2_n\rangle\vec{x} &= E_2(F)\vec{x} \\ \vdots &= \vdots \\ F \circ \langle t^k_1,\ldots,t^k_n\rangle\vec{x} &= E_k(F)\vec{x} \end{aligned}$$

   Pede-se:

   - Mostre pelo menos cinco exemplos já conhecidos de funções definidas desta forma,com suas repectivas definições;
   - Argumente que toda funçao primitiva recursiva pode-se ser definida de acordo com o esquema acima;
   - Argumente que toda função definida da forma acima é parcialmente recursiva;

7. Comente a necessidade ou não do uso do teorema da recursão na sua resposta questão acima.

8. (Opcional pontos) Na questão acima, verifique e depois argumente como fica uma prova de que recursão geral em n+1 argumentos define funções computáveis que não podem ser definidas com recursão sobre somente n argumentos.

9. (optional pontos) Em aula usamos a ordem lexicográgica sobre pares de números naturais para provar que a função de Ackerman, questão 5, é bem definida, ou seja, tem valor para todo (n,m). Também mostramos que esta função não é primitiva recursiva. Considere agora uma função definida segundo a definição abaixo com recursão sobre 3 variáveis.

$$H(0,0,n) = n+1 \quad (4)$$
$$H(0,m+1,0) = H(0,m,1) \quad (5)$$
$$H(0,m+1,n+1) = H(0,m,H(0,m+1,n)) \quad (6)$$
$$H(y+1,m+1,0) = H(y,m+1,1) \quad (7)$$
$$H(y+1,m+1,n+1) = H(y,H(y+1,m,n+1),H(y+1,m+1,n)) \quad (8)$$

Prove que H é bem definida. H pode ser definida com recursão sobre 2 variáveis ? Justifique.

10. Considere a frase "O λ-calculus e a Máquina de Turing são equivalentes". Formalize esta frase de forma que possa ser lida como um teorema matemático. Resposta em um quarto de página.

11. Questões de Lambda-calculo. Um λ-termo é normal se e somente se ele não possue nenhuma ocorrência de β-redex, i.e, um subtermo na forma $(\lambda x.M)N$. Marque (V) ou (F) justificando.

    1. $(\lambda h.(\lambda x.h(xx))(\lambda x.h(xx)))N$, onde N é um λ-termo, não tem forma normal
    2. Todos os tipos de dados apresentados em aula (Booleano, Lista, Numeros Naturais) são representados por termos em forma normal.
    3. Qualquer ordem de aplicações de β-redução a um λ-termo obtem sua formal normal.
    4. Considere Suc como o λ-termo que implementa a função de sucessor sobre números naturais na representação de numerais de Church. Seja Y o termo $(\lambda h.(\lambda x.h(xx))(\lambda x.h(xx)))$. O termo $Y(SUC)\underline{n}$, ou $YSUC\underline{n}$ não possui forma normal, onde $\underline{n}$ é um numeral de Church para o número n.

12. Prove que existe um λ-termo K, tal que, qualquer que seja o λ-termo M, KM reduz-se a K. Argumente como este termo K pode ser visto como o programa que imprime a si mesmo.

13. Indique como implementar arrays em λ-calculus. Tente fazer o mais simples possível. Indique somente como implementar arrays de uma dimensão, ou seja vetores.

14. Na questão acima, verifique se é possível implementar de forma sucinta arrays de mais de uma dimensão. O termo sucinto tem a ver com elegância, portanto esta questão tem algo de subjetivo, defina primeiro o que você entende por sucinto.

15. O combinador D, tal que Dx se reduz a xx, pode ser escrito como combinação de S e K. Prove o teorema $x \to (x \to x)$ no sistema axiomático e veja através desta prova como escrever D em função de S e K.

16. (opcional) Indique uma forma geral de obter a partir de uma fórmula proposicional só com implicações um combinador associado a ela indicado como é esta associação. Demonstre informalmente como para cada prova de teorema na lógica com os axiomas S e K e a regra de modus ponens, existe um combinador associado. Formalize esta associação. Usando este resultado você pode dizer se existe um combinador associado, segundo a associação estabelecida pelo seu resultado, à fórmula $(A \to (B \to C)) \to (B \to (A \to C))$? E a fórmula $((A \to B) \to C) \to ((B \to A) \to C)$?

BOA LISTA